Mini Project

1) System level code-

```
`timescale 1ns/1ps
module AHBLITE_SYS(
  //CLOCKS & RESET
  input    wire       CLK,
  input    wire       RESET,
  input    wire       HSEL,
  //TO BOARD LEDs
  output    wire  [7:0]  LED,


  // Debug
  //input    wire      TCK_SWCLK,         // SWD Clk / JTAG TCK
  input    wire       TDI_NC,          // NC    / JTAG TDI
  inout    wire       TMS_SWDIO,         // SWD I/O / JTAG TMS
  output    wire       TDO_SWO          // SW  Out / JTAG TDO
);
 //AHB-LITE SIGNALS
//Gloal Signals
wire               HCLK;
wire               HRESETn;
//Address, Control & Write Data Signals
wire [31:0]            HADDR;
wire [31:0]            HWDATA;
wire               HWRITE;
wire [1:0]            HTRANS;
wire [2:0]            HBURST;
wire               HMASTLOCK;
wire [3:0]            HPROT;
wire [2:0]            HSIZE;
//Transfer Response & Read Data Signals
wire [31:0]      HRDATA;
wire               HRESP;
wire               HREADY;

//SELECT SIGNALS
wire [3:0]            MUX_SEL;

wire                  HSEL_MEM;

wire                  HSEL_LED;
wire                  HSEL_TIMER;
wire                  HSEL_SINGLERAM;
/*assign HSEL_S0 = (HADDR[31:24] == 16'b0000_0000_00000001);
assign HSEL_S1  = (HADDR[31:24] ==16'b0000_0000_0000_0010);
```

```
assign HSEL_S3 = (HADDR[31:24] ==16'b0000_0000_0000_1000 );
assign HSEL_S6   = (HADDR[31:24] ==16'b0000_0000_0100_0000);
*/
//SLAVE READ DATA
wire [31:0]        HRDATA_S0;
//wire [31:0]      HRDATA_VGA;
//wire [31:0]      HRDATA_UART;
wire [31:0]        HRDATA_S1;
wire [31:0]        HRDATA_S3;
wire [31:0]        HRDATA_S6;

//SLAVE HREADYOUT
wire                            HREADYOUT_S0;
//wire                          HREADYOUT_VGA;
//wire                          HREADYOUT_UART;
wire                            HREADYOUT_S1;
wire                            HREADYOUT_S3;
wire                            HREADYOUT_S6;

//CM0-DS Sideband signals
wire [31:0]             IRQ;

// CM-DS Sideband signals
wire        lockup;
wire        lockup_reset_req;
wire        sys_reset_req;

//SYSTEM GENERATES NO ERROR RESPONSE
assign                  HRESP = 1'b0;

// Interrupt signals

wire        TIMER_IRQ;
assign       IRQ = 32'b00000000;


// Clock
wire        fclk;             // Free running clock
// Reset
wire        reset_n = RESET;
wire HSEL_MEM, HSEL_TIMER, HSEL_LED;


// Clock divider, divide the frequency by two, hence less time constraint
reg clk_div=1'b1;
always @(posedge CLK)
begin
   clk_div=~clk_div;
```

```
end
BUFG BUFG_CLK (
   .O(fclk),
   .I(clk_div)
);
// Reset synchronizer
reg  [4:0]     reset_sync_reg;
assign lockup_reset_req = 1'b0;
always @(posedge fclk or negedge reset_n)
begin
   if (!reset_n)
      reset_sync_reg <= 5'b00000;
   else
   begin
      reset_sync_reg[3:0] <= {reset_sync_reg[2:0], 1'b1};
      reset_sync_reg[4] <= reset_sync_reg[2] &
                        (~(sys_reset_req | (lockup & lockup_reset_req)));
   end
end

// CPU System Bus
assign HCLK = fclk;
assign HRESETn = reset_sync_reg[4];

// Debug signals (DesignStart Cortex-M0 supports only SWD)
wire dbg_swdo_en;
wire dbg_swdo;
wire dbg_swdi;
assign TMS_SWDIO = dbg_swdo_en ? dbg_swdo : 1'bz;
assign dbg_swdi = TMS_SWDIO;
wire cdbgpwrupreq2ack;

// DesignStart simplified integration level
CORTEXM0INTEGRATION u_CORTEXM0INTEGRATION (
   // CLOCK AND RESETS
   .FCLK         (fclk),
   .SCLK         (fclk),
   .HCLK         (fclk),
   .DCLK         (fclk),
   .PORESETn     (reset_sync_reg[2]),
   .DBGRESETn    (reset_sync_reg[3]),
   .HRESETn      (HRESETn),
   .SWCLKTCK     (TCK_SWCLK),
   .nTRST        (1'b1),

   // AHB-LITE MASTER PORT
   .HADDR        (HADDR),
   .HBURST       (HBURST),
```

```
.HMASTLOCK    (HMASTLOCK),
.HPROT        (HPROT),
.HSIZE        (HSIZE),
.HTRANS       (HTRANS),
.HWDATA       (HWDATA),
.HWRITE       (HWRITE),
.HRDATA       (HRDATA),
.HREADY       (HREADY),
.HRESP        (HRESP),
.HMASTER      (),

// CODE SEQUENTIALITY AND SPECULATION
.CODENSEQ     (),
.CODEHINTDE   (),
.SPECHTRANS   (),

// DEBUG
.SWDITMS      (dbg_swdi),
.TDI          (TDI_NC),
.SWDO         (dbg_swdo),
.SWDOEN       (dbg_swdo_en),
.TDO          (TDO_SWO),
.nTDOEN       (),
.DBGRESTART   (1'b0),
.DBGRESTARTED (),
.EDBGRQ       (1'b0),            // External Debug request to CPU
.HALTED       (),

// MISC
.NMI          (1'b0),            // Non-maskable interrupt input
.IRQ          (IRQ),             // Interrupt request inputs
.TXEV         (),                // Event output (SEV executed)
.RXEV         (1'b0),            // Event input
.LOCKUP       (lockup),          // Core is locked-up
.SYSRESETREQ  (sys_reset_req),   // System reset request
.STCALIB      ({1'b1,            // No alternative clock source
               1'b0,             // Exact multiple of 10ms from FCLK
               24'h007A11F}),    // Calibration value for SysTick for 50 MHz source
.STCLKEN      (1'b0),            // SysTick SCLK clock disable
.IRQLATENCY   (8'h00),
.ECOREVNUM    (28'h0),

// POWER MANAGEMENT
.GATEHCLK     (),
.SLEEPING     (),                // Core and NVIC sleeping
.SLEEPDEEP    (),
.WAKEUP       (),
.WICSENSE     (),
```

```
    .SLEEPHOLDREQn (1'b1),
    .SLEEPHOLDACKn (),
    .WICENREQ      (1'b0),
    .WICENACK      (),
    .CDBGPWRUPREQ  (cdbgpwrupreq2ack),
    .CDBGPWRUPACK  (cdbgpwrupreq2ack),

    // SCAN IO
    .SE            (1'b0),
    .RSTBYPASS     (1'b0)
);

//Address Decoder

AHBDCD uAHBDCD (
    .HADDR(HADDR[31:0]),

            .HSEL_S0(HSEL_MEM),
            .HSEL_S1(HSEL_LED),
    .HSEL_S2(),
    .HSEL_S3(HSEL_TIMER),
    .HSEL_S4(),
    .HSEL_S5(),
    .HSEL_S6(HSEL_SINGLERAM),
    .HSEL_S7(),
    .HSEL_S8(),
    .HSEL_S9(),
    .HSEL_NOMAP(),

            .MUX_SEL(MUX_SEL[3:0])
);

//Slave to Master Mulitplexor

AHBMUX uAHBMUX (
            .HCLK(HCLK),
            .HRESETn(HRESETn),
            .MUX_SEL(MUX_SEL[3:0]),

            .HRDATA_S0(HRDATA_MEM),
            .HRDATA_S1(HRDATA_LED),
            .HRDATA_S2(32'h00000000),
            .HRDATA_S3(HRDATA_TIMER),
            .HRDATA_S4(32'h00000000),
            .HRDATA_S5(32'h00000000),
            .HRDATA_S6(HRDATA_SINGLERAM),
            .HRDATA_S7(32'h00000000),
            .HRDATA_S8(32'h00000000),
```

```
        .HRDATA_S9(32'h00000000),
        .HRDATA_NOMAP(32'hDEADBEEF),

        .HREADYOUT_S0(HREADYOUT_MEM),
        .HREADYOUT_S1(HREADYOUT_LED),
        .HREADYOUT_S2(1'b1),
        .HREADYOUT_S3(HREADYOUT_TIMER),
        .HREADYOUT_S4(1'b1),
        .HREADYOUT_S5(1'b1),
        .HREADYOUT_S6(HREADYOUT_SINGLERAM),
        .HREADYOUT_S7(1'b1),
        .HREADYOUT_S8(1'b1),
        .HREADYOUT_S9(1'b1),
        .HREADYOUT_NOMAP(1'b1),

        .HRDATA(HRDATA[31:0]),
        .HREADY(HREADY)
);

// AHBLite Peripherals


// AHB-Lite RAM
AHB2MEM uAHB2MEM (
        //AHBLITE Signals
        .HSEL(HSEL_MEM),
        .HCLK(HCLK),
        .HRESETn(HRESETn),
        .HREADY(HREADY_MEM),
        .HADDR(HADDR),
        .HTRANS(HTRANS[1:0]),
        .HWRITE(HWRITE),
        .HSIZE(HSIZE),
        .HWDATA(HWDATA[31:0]),

        .HRDATA(HRDATA_MEM),
        .HREADYOUT(HREADYOUT_MEM)
);

// AHBLite timer
AHBTIMER uAHBTIMER(
        .HCLK(HCLK),
        .HRESETn(HRESETn),
        .HADDR(HADDR),
        .HWDATA(HWDATA),
        .HREADY(HREADY_TIMER),
        .HWRITE(HWRITE),
        .HTRANS(HTRANS),
```

```
            .HSEL(HSEL_TIMER),
            .HRDATA(HRDATA_TIMER),
            .HREADYOUT(HREADYOUT_TIMER),

            .timer_irq(TIMER_IRQ) //this statement is commented in timer only
            );

AHBSINGLERAM uAHBSINGLERAM (
   //AHBLITE Signals
   .CS(HSEL_SINGLERAM),
   .CLK(HCLK),
   .HRESETn(HRESETn),
   .HREADY(HREADY_SINGLERAM),
   .HADDR(HADDR),
   .HTRANS(HTRANS),
   .WE(HWRITE),
   .HSIZE(HSIZE),
   .DATA_IN(DATA_IN),

   .HRDATA(HRDATA_SINGLERAM),
   .HREADYOUT(HREADYOUT_SINGLERAM)

);
AHB2LED uAHB2LED (
   //AHBLITE Signals
   .HSEL(HSEL_LED),
   .HCLK(HCLK),
   .HRESETn(HRESETn),
   .HREADY(HREADY_LED),
   .HADDR(HADDR),
   .HTRANS(HTRANS),
   .HWRITE(HWRITE),
   .HSIZE(HSIZE),
   .HWDATA(HWDATA),

   .HRDATA(HRDATA_LED),
   .HREADYOUT(HREADYOUT_LED),

   .LED(LED[7:0])
);


endmodule
```

2) Assembly code-

```
                                PRESERVE8
                THUMB


; Vector Table Mapped to Address 0 at Reset
                AREA    RESET, DATA, READONLY   ; First 32 WORDS is
VECTOR TABLE
                EXPORT  __Vectors

__Vectors       DCD     0x00003FFC
                DCD     Reset_Handler
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0

                ; External Interrupts
                DCD     Timer_Handler
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0


                AREA    |.text|, CODE, READONLY

; Reset Handler
Reset_Handler   PROC
                GLOBAL  Reset_Handler
                ENTRY
```

```
                LDR     R1, =0xE000E100         ; Interrupt Set Enable
Register
                LDR     R0, =0x00000001         ; Enable interrupts for
timer.  If UARtimer
                STR     R0, [R1]

                ; Configure the timer
                LDR     R1, =0x52000000         ; Timer load value
register
                LDR     R0, =0x0F       ; = 50,000,000 (system tick
frequency)
                STR     R0, [R1]
                LDR     R1, =0x52000008         ; Timer control
register
                MOVS    R0, #0x07               ; Prescaler = 1, enable
timer, reload mode
                STR     R0, [R1]



AGAIN           NOP
                B       AGAIN

                ENDP

Timer_Handler   PROC
                EXPORT  Timer_Handler
                       PUSH    {LR}

                LDR     R1, =0x5200000c         ; Clear timer interrupt
                MOVS    R0, #0x01               ; CLEAR REGISTER HIGH.
means timer_IRQ will be cleared to 0
                STR     R0, [R1]
                LDR     R0,=0xFF
                STR     R0,[R1]
                LDR     R0, =0x
                PRESERVE8
                THUMB


; Vector Table Mapped to Address 0 at Reset
                AREA    RESET, DATA, READONLY   ; First 32 WORDS is
VECTOR TABLE
                EXPORT  __Vectors

__Vectors       DCD     0x00003FFC
                DCD     Reset_Handler
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
```

```
                DCD     0
                DCD     0
                DCD     0

                ; External Interrupts
                DCD     Timer_Handler
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0
                DCD     0


                AREA    |.text|, CODE, READONLY

; Reset Handler
Reset_Handler   PROC
                GLOBAL  Reset_Handler
                ENTRY


                ; Configure the timer

                LDR     R1, =0xE000E100         ; Timer load value
register
                LDR     R0, =0x00000001
                STR     R0, [R1]


;           LDR       R0, =0x10                      ;Delay
;Loop       SUBS  R0,R0,#1
            ; BNE     Loop
                LDR     R1, =0x52000000
                LDR     R0, =0x0F
                STR     R0, [R1]
                LDR     R1, =0x52000008
                                                       ; Timer control
register
                MOVS    R0, #0x07               ; Set prescaler, reload
mode, start timer
                STR     R0, [R1]
AGAIN           NOP
                B       AGAIN


                ENDP

Timer_Handler  PROC
```

```
                EXPORT  Timer_Handler
                                PUSH    {LR}
                LDR     R1, =0x5200000c
                MOVS    R0,  #0x01
                STR     R0, [R1]
                       MOVS    R0,  #0x00

                STR     R0, [R1]
                       LDR     R1, =0x50000000   ;LED
                LDR     R0, =0xFF
                STR     R0, [R1]
                   LDR      R0, =0x5                    ;Delay
Loop1         SUBS     R0,R0,#1
                       BNE     Loop1
                LDR    R1, =0x50000000                  ;Write to LED
with value
           LDR   R0, =0x00
           STR   R0, [R1]


               POP     {PC}
                ENDP

                ENDP

                ALIGN     4


                END
```

3) Testbench-

```
`timescale 1 ns / 1 ps
module tb();
reg clk,rst;
wire [7:0] LED;
//reg [7:0] SW;

//wire test_bit = tb.dut.u_CORTEXM0INTEGRATION.u_cortexm0.cm0_r00[2];

//assign tb.dut.u_CORTEXM0INTEGRATION.u_cortexm0.cm0_r00 = (test_bit==1'b0) ?
32'd0:tb.dut.u_CORTEXM0INTEGRATION.u_cortexm0.cm0_r00;


AHBLITE_SYS dut(.CLK(clk),
        .RESET(rst),
        //.SW(SW),
        //.TCK_SWCLK(),
        //.TDI_NC(),
        //.TMS_SWDIO(),
       // .TDO_SWO(),
       .LED(LED)
        );
```

```
initial
   begin
      rst=0;
      clk=0;
      #7;
      rst=1;
   end

always #5 clk=~clk;

initial
   begin
      $dumpfile ("AHBLITE_SYS.vcd");
      $dumpvars (0, tb);
      #1;
   end

endmodule
```

4) Waveforms-